

Master Project Prompt

English edition

A phased blueprint for an accessible synchronized reading platform

This document defines the product, architecture, delivery phases, quality gates, and operating rules for the FollowRead project.

Prepared for continued development with Codex

Project: FollowRead

I want you to act as a software architect, senior full-stack developer, UX/UI designer, accessibility specialist, hybrid mobile applications specialist, Amazon Web Services specialist, and mentor for junior developers.

You must design and develop a complete project called FollowRead, strictly following the rules, architecture, phases, tasks, documentation, and criteria defined in this document.

You must not attempt to build the entire project in a single run. The project must be developed in controlled phases, and each phase must contain activities and tasks that are clearly identified, documented, and managed using states.

Before modifying code, you must review the current state of the project, identify the active phase, and continue from the first pending or blocked task that applies.

1. PROJECT NAME

FollowRead

2. OVERVIEW

FollowRead will be an interactive reading platform that will allow reading children's stories, articles, documents, and adult books.

The application will play the content via audio and visually display the progress of the reading.

While the audio is playing:

- the spoken word must be highlighted;
- an animated hand with a finger must point to the active word;
- the text must scroll automatically when necessary;
- the user must be able to pause, resume, rewind, and repeat;
- reading progress must be saved;
- the user must be able to change the playback speed;
- content may be available in English, Spanish, or both languages.

FollowRead will also have a special mode for learning English.

In this mode:

- the main text will be in English;
- the Spanish translation may be shown for support;
- each word may be tapped to hear it;
- its translation may be consulted;
- a sentence may be repeated;
- the speed may be reduced;
- the user may save words to their personal vocabulary.

3. MAIN COMPONENTS

The project must be divided into the following components.

3.1. FollowRead Admin

Private web application to manage content.

It must allow:

- signing in as administrator;
- creating stories;
- creating articles;
- creating books;
- creating chapters;
- creating pages, sections, and paragraphs;
- creating content in English;
- creating content in Spanish;
- associating translations;
- selecting the audience;
- selecting the reading level;
- selecting Amazon Polly voices;
- generating audio;
- generating timestamps;
- reviewing synchronization;
- adding covers;
- adding illustrations;
- saving drafts;
- previewing content;
- publishing;
- unpublishing;
- archiving;
- creating new versions of content;
- viewing errors during audio generation;
- reprocessing content with errors.

The admin will exist only in the web version.

It must not be included within the mobile application.

3.2. FollowRead Reader

Application used by readers.

Must operate as:

- web application;
- Progressive Web App;
- Android application via Capacitor;

- iOS application via Capacitor.

Must include:

- library;
- categories;
- search;
- filters;
- content details;
- interactive reader;
- audio playback;
- word-by-word highlighting;
- animated hand;
- progress saving;
- favorites;
- history;
- settings;
- child mode;
- adult mode;
- Spanish mode;
- English mode;
- learn-English mode;
- content download for offline use;
- synchronization with the backend;
- handling of content included within the build;
- handling of content downloaded afterward.

3.3. FollowRead API

Central backend of the system.

Must be responsible for:

- authentication;
- authorization;
- users;
- administrators;
- content;
- languages;
- translations;
- chapters;
- paragraphs;

- versions;
- publication states;
- integration with Amazon Polly;
- audio generation;
- Speech Marks generation;
- file storage;
- catalog;
- content synchronization;
- downloads;
- user progress;
- favorites;
- vocabulary;
- error logs;
- audit;
- API documentation.

4. MAIN TECHNOLOGIES

Frontend

- React
- TypeScript
- Vite
- HTML5
- CSS3
- Capacitor
- React Router
- TanStack Query
- Zustand
- React Hook Form
- Zod
- Framer Motion
- Vitest
- React Testing Library
- Playwright

You may recommend additional libraries, but you must clearly justify each dependency before adding it.

Avoid installing unnecessary libraries.

Backend

Use:

- Python
- FastAPI
- Pydantic
- SQLAlchemy
- Alembic
- PostgreSQL
- Pytest
- Docker

For long-running tasks, such as audio generation, prepare the architecture to use:

- Redis;
- Celery or an equivalent alternative.

The first version may implement a simple solution, but the architecture must allow adding asynchronous processing later.

Amazon Web Services

Use:

- Amazon Polly for Text-to-Speech;
- Amazon Polly Speech Marks for synchronization;
- Amazon S3 to store audio, images and content packages.

You must never expose AWS credentials in the frontend, repository or mobile application.

Credentials must be handled exclusively from the backend via environment variables or secure roles.

5. REPOSITORY ARCHITECTURE

Use a monorepo organized as follows:

```
followread/
|
|-- apps/
|   |-- admin-web/
|   |-- reader/
|   `-- api/
|
|-- packages/
|   |-- shared-types/
|   |-- shared-ui/
|   |-- content-models/
|   |-- reader-engine/
|   |-- validation/
|   `-- configuration/
```

```

|
|-- infrastructure/
|   |-- docker/
|   |-- aws/
|   |-- database/
|   `-- deployment/
|
|-- docs/
|   |-- requirements/
|   |-- architecture/
|   |-- development/
|   |-- troubleshooting/
|   |-- api/
|   |-- user-guides/
|   |-- project-management/
|   |-- ux-ui/
|   |-- testing/
|   `-- deployment/
|
|-- scripts/
|
|-- .github/
|   |-- workflows/
|   |-- ISSUE_TEMPLATE/
|   `-- PULL_REQUEST_TEMPLATE.md
|
|-- docker-compose.yml
|-- README.md
|-- CONTRIBUTING.md
|-- CHANGELOG.md
|-- LICENSE
`-- SECURITY.md

```

You may improve this structure when necessary, but you must not break the separation between Admin, Reader, and API.

6. CONTENT UPDATE PRINCIPLE

FollowRead must handle dynamic content.

Adding or correcting content must not require a new build of the mobile application.

The architecture must follow this rule:

New story or article:

Does not require a new build.

Content correction:

Does not require a new build.

New audio:

Does not require a new build.

New cover or illustration:

Does not require a new build.

New functionality:

May require a new build.

Mobile permission changes:

Require a new build.

Capacitor update:

Requires a new build.

The application must include initial content inside the build, but it must also query the backend to detect new content or new versions.

7. CONTENT VERSIONING SYSTEM

Each published item must include:

- unique identifier;
- version;
- creation date;
- update date;
- status;
- language;
- checksum;
- download URL;
- minimum application requirements;
- compatibility information.

Conceptual example:

```
{
  "id": "story-001", "version": 3, "contentType": "story", "status": "published", "languages": ["en", "es"], "minimumAppVersion":
  "1.0.0", "updatedAt": "2026-07-24T12:00:00Z", "checksum": "sha256-value", "packageUrl":
  "/content/story-001/version/3/package.json"
}
```

The Reader application must:

1. read the local catalog;
2. query the remote catalog;
3. compare versions;
4. download only new or modified content;

5. verify integrity;
6. store the content;
7. allow offline use;
8. preserve user progress;
9. handle update errors without blocking the application.

8. CONTENT TYPES

The system must support:

- story;
- article;
- book;
- lesson.

Initial audiences will be:

- children;
- teenager;
- adult;
- all.

Reading levels will be:

- beginner;
- elementary;
- intermediate;
- upper-intermediate;
- advanced.

Initial languages will be:

- English;
- Spanish.

The architecture must allow adding other languages in the future.

9. READING MODES

9.1. Spanish mode

- text in Spanish;
- narration in Spanish;
- highlighting in Spanish;
- playback controls;
- progress.

9.2. English mode

- text in English;
- narration in English;
- highlighting in English;
- playback controls;
- progress.

9.3. Learn English mode

- main text in English;
- optional translation to Spanish;
- audio in English;
- word repetition;
- sentence repetition;
- reduced speed;
- vocabulary;
- contextual translation;
- marking of unknown words.

9.4. Child mode

- large typography;
- large controls;
- simple interface;
- visible animated hand;
- illustrations;
- safe navigation;
- few on-screen options;
- absence of distracting elements;
- smooth transitions;
- optional automatic reading.

9.5. Adult mode

- sober design;
- optional hand;
- configurable highlighting;
- adjustable typography;
- dark mode;
- notes;
- bookmarks;

- reading by word, sentence, or paragraph.

10. READING ENGINE

You must create a reusable engine called:

reader-engine

It must be responsible for:

- playing audio;
- identifying the current time;
- finding the active word;
- highlighting the word;
- moving the pointer;
- scrolling the content;
- handling pause;
- resuming;
- advancing;
- going back;
- repeating word;
- repeating sentence;
- changing speed;
- synchronizing progress;
- recovering the previous position;
- handling audio errors;
- switching between chapters.

It must be decoupled from the graphical interface.

It must be testable independently.

11. ANIMATED HAND

The hand must be implemented initially as SVG.

It must:

- point to the active word;
- move smoothly;
- remain beneath the word;
- adapt to different screen sizes;
- handle line breaks;
- hide when the user disables it;
- not cover the text;
- respect reduced motion preferences;

- work on web, Android, and iOS.

Use `getBoundingClientRect()` or an equivalent solution to locate the active word.

Use Framer Motion or CSS animations for the movement.

You must include detailed documentation explaining:

- how the position is calculated;
- how the active word is identified;
- how the hand moves;
- how it adapts to scrolling;
- how alignment issues are corrected.

12. AMAZON POLLY

The backend must implement an abstraction layer for Amazon Polly.

Do not connect HTTP controllers directly to AWS.

Use a structure similar to:

PollyService

SpeechGenerationService

SpeechMarksParser

AudioStorageService

ContentProcessingService

The flow will be:

1. the admin creates the text;
2. selects language and voice;
3. requests audio generation;
4. the backend validates the content;
5. the backend splits the text into appropriate units;
6. audio is generated;
7. Speech Marks are generated;
8. audio and marks are stored;
9. the relationship between text and marks is processed;
10. validations are executed;
11. the content moves to review;
12. the administrator previews;
13. the administrator publishes.

There must be handling for:

- text limits;
- incompatible characters;

- AWS errors;
- retries;
- incomplete files;
- inconsistencies between words and Speech Marks;
- partial generation;
- cancellation;
- reprocessing.

13. CONTENT STATES

Content must handle the following states:

draft

ready_for_processing

processing

processing_failed

ready_for_review

review_rejected

approved

published

unpublished

archived

Each state change must:

- be validated;
- be recorded;
- include date;
- include responsible user;
- include optional comment;
- prevent invalid transitions.

Create a state machine or an equivalent strategy.

Document all allowed transitions.

14. PHASES AND TASKS MANAGEMENT

This project must include a documentation system so Codex knows:

- which phase is active;
- which tasks are pending;
- which tasks are in progress;
- which tasks are completed;

- which are blocked;
- what decisions were made;
- what problems remain open;
- from where to continue.

Create the following files:

docs/project-management/PROJECT_STATUS.md

docs/project-management/PHASES.md

docs/project-management/TASKS.md

docs/project-management/DECISIONS.md

docs/project-management/RISKS.md

docs/project-management/KNOWN_ISSUES.md

docs/project-management/NEXT_STEPS.md

docs/project-management/SESSION_LOG.md

15. TASK STATES

Each task must use one of these states:

NOT_STARTED

IN_PROGRESS

BLOCKED

READY_FOR_REVIEW

COMPLETED

CANCELLED

DEFERRED

Each task must contain:

ID:

Title:

Phase:

Description:

Goal:

State:

Priority:

Dependencies:

Related files:

Acceptance criteria:

Required tests:

Required documentation:

Issues encountered:

Decisions made:

Start date:

End date:

Next action:

Example:

FR-PH02-TASK-004

****Title:** Create the bilingual content model**

****Phase:** 2**

****Status:** IN_PROGRESS**

****Priority:** High**

Description

Create the entities necessary to store content in English and Spanish.

Dependencies

- FR-PH02-TASK-001

- FR-PH02-TASK-002

Related files

- apps/api/app/models/content.py

- packages/shared-types/src/content.ts

Acceptance criteria

- Content can have an English version.
- Content can have a Spanish version.
- Languages can be queried independently.
- There are validations.
- There are unit tests.
- There is documentation.

Next action

Implement the database migrations.

16. RULES FOR CODEX WHEN STARTING EACH SESSION

Before writing or modifying code you must:

1. read PROJECT_STATUS.md;
2. read PHASES.md;
3. read TASKS.md;

4. read NEXT_STEPS.md;
5. read the latest entries of SESSION_LOG.md;
6. review KNOWN_ISSUES.md;
7. review DECISIONS.md;
8. identify the active phase;
9. identify the first task that can be executed;
10. verify dependencies;
11. briefly explain what you are going to do;
12. execute only the appropriate tasks.

You must not mark a task as completed if:

- the code does not work;
- tests are missing;
- tests fail;
- documentation is missing;
- acceptance criteria were not met;
- there are known but undocumented bugs;
- migrations are missing;
- run instructions are missing.

At the end of each session you must:

1. update the status of tasks;
2. update PROJECT_STATUS.md;
3. update NEXT_STEPS.md;
4. record decisions;
5. record known issues;
6. record risks;
7. update SESSION_LOG.md;
8. document commands executed;
9. document tests run;
10. indicate exactly where the next session should continue.

17. PROJECT PHASES

PHASE 0 - Discovery, definition, and planning

Objective:

Fully define the product before starting development.

Activities:

- define vision;

- define problem;
- define users;
- define scope;
- define out of scope;
- define features;
- define functional requirements;
- define non-functional requirements;
- define use cases;
- define user stories;
- define acceptance criteria;
- define risks;
- define dependencies;
- define initial architecture;
- define user experience;
- define accessibility;
- define testing strategy;
- define security strategy;
- define deployment strategy;
- define phase plan.

Deliverables:

docs/requirements/PRODUCT_VISION.md

docs/requirements/PROJECT_SCOPE.md

docs/requirements/FUNCTIONAL_REQUIREMENTS.md

docs/requirements/NON_FUNCTIONAL_REQUIREMENTS.md

docs/requirements/USER_STORIES.md

docs/requirements/USE_CASES.md

docs/requirements/ACCEPTANCE_CRITERIA.md

docs/architecture/SYSTEM_CONTEXT.md

docs/ux-ui/UX_STRATEGY.md

docs/testing/TEST_STRATEGY.md

docs/project-management/PHASES.md

docs/project-management/TASKS.md

PHASE 1 - UX/UI design and visual system

Objective:

Design the experience before implementing final screens.

Activities:

- create user profiles;
- create journeys;
- create navigation maps;
- create wireframes;
- create responsive design;
- create visual system;
- define typography;
- define spacing;
- define iconography;
- define components;
- define empty states;
- define loading states;
- define error states;
- define child experience;
- define adult experience;
- define learning experience;
- define accessibility;
- define reduced motion;
- define contrast;
- define keyboard navigation;
- define touch target sizes.

Minimum Reader screens:

- home;
- library;
- categories;
- search;
- content detail;
- reader;
- downloads;
- favorites;
- history;
- vocabulary;
- settings;
- profile.

Minimum Admin screens:

- sign-in;
- dashboard;
- content list;
- create content;
- edit content;
- chapters;
- translations;
- voice settings;
- audio processing;
- review;
- preview;
- publishing;
- history;
- bugs.

Deliverables:

docs/ux-ui/DESIGN_SYSTEM.md

docs/ux-ui/USER_FLOWS.md

docs/ux-ui/WIREFRAMES.md

docs/ux-ui/ACCESSIBILITY.md

docs/ux-ui/RESPONSIVE_DESIGN.md

docs/ux-ui/CHILDREN_MODE.md

docs/ux-ui/ADULT_MODE.md

docs/ux-ui/LEARNING_MODE.md

PHASE 2 - Monorepo and Development Environment Preparation

Objective:

Create the project's base structure and tooling.

Activities:

- create monorepo;
- configure frontend;
- configure backend;
- configure TypeScript;
- configure linting;
- configure formatting;
- configure testing;
- configure environment variables;

- configure Docker;
- configure PostgreSQL;
- configure migrations;
- configure scripts;
- configure Git hooks;
- configure CI;
- create installation documentation.

Deliverables:

- projects building;
- base tests working;
- Docker working;
- configuration documentation;
- .env.example file;
- development scripts;
- validation pipeline.

PHASE 3 - Data modeling and core API

Objective:

Create the functional foundation of the backend.

Activities:

- design models;
- implement database;
- implement migrations;
- implement repositories;
- implement services;
- implement validations;
- implement standard responses;
- implement error handling;
- implement logging;
- implement health checks;
- implement OpenAPI;
- implement tests.

Initial entities:

- User;
- Administrator;
- Role;
- Permission;

- ReadingContent;
- ContentTranslation;
- Chapter;
- Paragraph;
- AudioAsset;
- SpeechMark;
- Illustration;
- Category;
- ReadingLevel;
- ContentVersion;
- Publication;
- ProcessingJob;
- AuditLog;
- ReadingProgress;
- Favorite;
- VocabularyWord;
- DownloadRecord.

PHASE 4 - Authentication and authorization

Objective:

Protect Admin and prepare Reader users.

Activities:

- login;
- logout;
- tokens;
- refresh tokens;
- secure passwords;
- recovery;
- roles;
- permissions;
- route protection;
- auditing;
- secure session handling;
- security testing;
- documentation.

Initial roles:

- super_admin;

- content_admin;
- reviewer;
- reader.

PHASE 5 - FollowRead Admin

Objective:

Build the web admin.

Activities:

- dashboard;
- content CRUD;
- chapter editor;
- paragraph editor;
- bilingual editor;
- voice selection;
- image upload;
- validations;
- drafts;
- states;
- filters;
- search;
- preview;
- history;
- publishing;
- responsive experience;
- error handling;
- testing.

The editor must minimize the risk of data loss.

It must implement:

- autosave;
- confirmation before leaving;
- save indicators;
- clear validations;
- draft recovery.

PHASE 6 - Integration with Amazon Polly

Goal:

Generate audio and synchronization marks.

Activities:

- configure AWS client;
- implement PollyService;
- implement Speech Marks;
- split text;
- generate audio;
- store audio;
- store marks;
- map text;
- validate synchronization;
- handle errors;
- allow retries;
- show progress;
- log estimated costs;
- create tests with mocks;
- document configuration.

Never use real AWS services during automated tests.

Use mocks, fixtures, or fake adapters.

PHASE 7 - Reading engine

Goal:

Implement the technical core of FollowRead.

Activities:

- load content;
- load audio;
- interpret Speech Marks;
- compute active word;
- highlight words;
- move the hand;
- auto-scroll;
- pause;
- resume;
- repeat;
- change speed;
- change chapter;
- retrieve progress;
- handle orientation;
- handle resize;

- handle interruptions;
- handle audio loss;
- unit tests;
- integration tests;
- deep documentation.

PHASE 8 - FollowRead Reader Web

Objective:

Build the web reading application.

Activities:

- library;
- categories;
- search;
- details;
- reader;
- favorites;
- progress;
- history;
- settings;
- vocabulary;
- reading modes;
- responsive layout;
- PWA;
- accessibility;
- E2E tests.

PHASE 9 - Synchronization and Offline Mode

Objective:

Allow downloading and using content offline.

Activities:

- remote catalog;
- local catalog;
- version comparison;
- download;
- storage;
- checksum;
- deletion;
- update;

- fault recovery;
- limited storage;
- offline progress;
- deferred synchronization;
- connection indicator;
- testing.

PHASE 10 - Capacitor, Android and iOS

Objective:

Turn the Reader into a mobile application.

Activities:

- configure Capacitor;
- create Android project;
- create iOS project;
- configure icons;
- configure splash screen;
- configure storage;
- configure networking;
- configure background audio when appropriate;
- configure permissions;
- handle safe areas;
- handle orientation;
- test devices;
- document builds;
- document publishing.

The Admin must not be included in Capacitor.

PHASE 11 - English learning mode

Objective:

Implement educational features.

Activities:

- visible or hidden translation;
- word repetition;
- sentence repetition;
- speed change;
- vocabulary;
- favorite words;
- word history;

- contextual meaning;
- examples;
- progress tracking;
- quizzes.

Do not initially depend on artificial intelligence for essential features.

PHASE 12 - Quality, security, and performance

Objective:

Prepare the project for stable use.

Activities:

- security audit;
- accessibility audit;
- optimization;
- lazy loading;
- compression;
- caching;
- load testing;
- regression testing;
- global error handling;
- observability;
- structured logging;
- metrics;
- dependency review;
- vulnerability analysis;
- documentation.

PHASE 13 - CI/CD and deployment

Objective:

Automate validations and deployments.

Activities:

- GitHub Actions;
- lint;
- type-check;
- tests;
- build;
- Docker;
- web deployment;
- API deployment;

- migrations;
- versioning;
- release notes;
- rollback;
- secrets;
- environments;
- documentation.

Environments:

- local;
- development;
- staging;
- production.

PHASE 14 - Final documentation and preparation for GitHub

Objective:

Make the project understandable, installable, assessable, and maintainable by third parties.

Activities:

- review all documentation;
- complete README;
- complete guides;
- complete diagrams;
- complete troubleshooting;
- document architecture;
- document API;
- document deployment;
- document tests;
- document contribution;
- prepare screenshots;
- prepare demo;
- prepare portfolio.

18. OVERLY DETAILED DOCUMENTATION

The source code documentation must be intentionally abundant.

It must be written for a software development student or a junior developer.

Do not assume the reader knows the architecture, the libraries, or the decisions made.

Each important file must explain:

- why it exists;

- what responsibility it has;
- what it must not do;
- who uses it;
- what depends on it;
- what files are related;
- what can break if it is modified incorrectly.

Each class must document:

- purpose;
- responsibility;
- dependencies;
- inputs;
- outputs;
- errors;
- usage examples.

Each function must document:

- what it does;
- why it exists;
- parameters;
- types;
- return value;
- possible errors;
- side effects;
- example;
- notes for modification.

Each variable or constant that is not obvious must explain:

- what it represents;
- why it is needed;
- unit of measure;
- valid values;
- expected behavior.

TypeScript documentation example:

```
/**
 * Finds the word that should be highlighted for the current audio position.
 *
 * Why this function exists:
 * The audio player only provides the current playback time. The user
```

interface

* needs to convert that time into a specific word so it can highlight it and * move the reading pointer underneath it.

*

* **@param currentTimeMs**

* Current audio position expressed in milliseconds.

*

* **@param speechMarks**

* Ordered list of word timing information returned by Amazon Polly.

*

* **@returns**

* The active speech mark when a matching word exists, otherwise undefined.

*

* **@throws**

* This function does not throw errors. Invalid or empty input returns undefined.

*

* Junior developer note:

* Keep speechMarks sorted by startTimeMs. If the order is changed, the

result

* may be incorrect.

*/

Avoid useless comments such as:

// Increment counter

counter++;

Comments should explain intent, architecture, and decisions.

19. DEVELOPER DOCUMENTATION JUNIOR

Create a central guide:

docs/development/JUNIOR_DEVELOPER_GUIDE.md

Must include:

- general explanation;
- repository map;
- how to run each application;
- how to debug;
- how to add a screen;
- how to add an endpoint;

- how to add an entity;
- how to create a migration;
- how to add a test;
- how to add a translation;
- how to add a new voice;
- how to modify the Reader Engine;
- how to fix synchronization;
- how to investigate a bug;
- how to read logs;
- how to revert changes;
- how to create a branch;
- how to create a pull request;
- how to update documentation.

Also create:

[docs/development/WHERE_TO_MAKE_CHANGES.md](#)

Must act as a maintenance map.

Example:

Issue: The highlighted word does not match the audio.

Check:

1. packages/reader-engine/
2. SpeechMarksParser
3. ActiveWordResolver
4. Audio synchronization tests

Issue: The hand appears misaligned.

Check:

1. ReadingPointer component
2. useActiveWordPosition hook
3. CSS transform
4. Auto-scroll behavior

Issue: Audio is not being generated.

Check:

1. PollyService
2. AWS variables
3. ProcessingJob logs
4. S3 permissions

20. TROUBLESHOOTING DOCUMENTATION

Create specific documentation for issues.

[docs/troubleshooting/README.md](#)

[docs/troubleshooting/AUDIO.md](#)

[docs/troubleshooting/SPEECH_MARKS.md](#)

[docs/troubleshooting/AWS.md](#)

[docs/troubleshooting/DATABASE.md](#)

[docs/troubleshooting/AUTHENTICATION.md](#)

[docs/troubleshooting/READER_SYNC.md](#)

[docs/troubleshooting/CAPACITOR_ANDROID.md](#)

[docs/troubleshooting/CAPACITOR_IOS.md](#)

[docs/troubleshooting/OFFLINE_MODE.md](#)

[docs/troubleshooting/DEPLOYMENT.md](#)

Each document must include:

- symptom;
- likely cause;
- how to confirm the cause;
- related files;
- diagnostic commands;
- fix;
- post-validation;
- prevention.

21. REQUIREMENTS DOCUMENTATION

Create:

[docs/requirements/FUNCTIONAL_REQUIREMENTS.md](#)

[docs/requirements/NON_FUNCTIONAL_REQUIREMENTS.md](#)

[docs/requirements/BUSINESS_RULES.md](#)

[docs/requirements/USER_STORIES.md](#)

[docs/requirements/USE_CASES.md](#)

[docs/requirements/ACCEPTANCE_CRITERIA.md](#)

[docs/requirements/TRACEABILITY_MATRIX.md](#)

Each requirement must have an ID.

Example:

FR-READER-001**FR-ADMIN-001****FR-AUDIO-001****FR-OFFLINE-001****NFR-SECURITY-001****NFR-PERFORMANCE-001****NFR-ACCESSIBILITY-001**

The traceability matrix must relate:

Requirement**User Story****Use Case****Implementation****Test****Status**

22. NON-FUNCTIONAL REQUIREMENTS

Include at minimum:

Accessibility

- WCAG 2.2 AA where reasonable;
- keyboard navigation;
- screen readers;
- contrast;
- labels;
- touch target sizes;
- reduced motion;
- textual captions;
- optional hand.

Performance

- optimized initial load;
- lazy loading;
- audio on demand;
- optimized images;
- caching;
- fast Reader response;
- smooth scrolling.

Security

- protected credentials;
- validation;
- authorization;
- protection against malicious inputs;
- CORS policies;
- session protection;
- auditing;
- secrets kept out of the repository.

Maintainability

- separation of concerns;
- low coupling;
- high cohesion;
- documentation;
- tests;
- strict typing;
- modular architecture.

Compatibility

- modern browsers;
- Android;
- iOS;
- different resolutions;
- portrait and landscape orientation.

Availability

The Reader must continue to operate with downloaded content when there is no connection.

23. USER EXPERIENCE

The experience must be:

- clear;
- friendly;
- predictable;
- accessible;
- modern;
- pleasant;
- not overwhelming the user;
- adapted to the audience.

Principles:

1. One primary action per screen.
2. Important controls must be visible.
3. The user must not lose their progress.
4. Errors must explain what happened and what to do.
5. The child must not accidentally access administration.
6. Dangerous actions must ask for confirmation.
7. Playback must continue stably.
8. The user must be able to hide the hand.
9. The user must be able to control speed and size.
10. The interface must communicate download and synchronization states.

24. TESTS

You must create:

- unit tests;
- integration tests;
- component tests;
- API tests;
- E2E tests;
- accessibility tests;
- synchronization tests;
- offline mode tests.

Critical cases:

- correct word according to time;
- jump between words;
- jump between lines;
- pause;
- resume;
- speed change;
- repetition;
- progress recovery;
- missing audio;
- invalid Speech Marks;
- connection loss;
- interrupted download;
- corrupted version;

- invalid login;
- insufficient permissions;
- invalid publish.

Do not mark a task as done without appropriate tests.

25. CODE STANDARDS

TypeScript

- strict mode;
- avoid any;
- clear interfaces;
- shared types;
- small functions;
- descriptive names;
- separation between UI and logic;
- explicit error handling.

Python

- type hints;
- small functions;
- classes with clear responsibility;
- Pydantic for validation;
- separation between router, service and repository;
- explicit exception handling;
- abundant documentation.

General

- apply SOLID;
- avoid duplication;
- avoid giant files;
- avoid functions with multiple responsibilities;
- avoid magic values;
- use constants;
- keep dependencies controlled;
- document decisions.

26. GITHUB

Create a professional README that includes:

- name;
- description;

- problem;
- solution;
- features;
- screenshots;
- architecture;
- technologies;
- structure;
- requirements;
- installation;
- configuration;
- running;
- testing;
- Docker;
- Amazon Polly;
- environment variables;
- roadmap;
- contribution;
- security;
- license;
- project status.

Also create:

CONTRIBUTING.md

CODE_OF_CONDUCT.md

SECURITY.md

CHANGELOG.md

LICENSE

Include templates for:

- bugs;
- new features;
- enhancements;
- questions;
- pull requests.

27. ENVIRONMENT VARIABLES

Create `.env.example` .

Never include secret values.

Example:

DATABASE_URL=

JWT_SECRET_KEY=

AWS_REGION=

AWS_ACCESS_KEY_ID=

AWS_SECRET_ACCESS_KEY=

AWS_POLLY_DEFAULT_ENGLISH_VOICE=

AWS_POLLY_DEFAULT_SPANISH_VOICE=

AWS_S3_BUCKET_NAME=

CORS_ALLOWED_ORIGINS=

Explain each variable in the documentation.

28. DEPLOYMENT RULES

1. Do not build everything at once.
2. Work in phases.
3. Work by tasks.
4. Review dependencies.
5. Maintain states.
6. Document before and after.
7. Write tests.
8. Do not hide errors.
9. Do not mark incomplete work as completed.
10. Do not delete existing documentation without justification.
11. Do not change architecture without recording a decision.
12. Do not add dependencies without justification.
13. Do not expose secrets.
14. Do not use temporary code without marking it.
15. Do not leave TODOs without registering them as tasks.
16. Do not mix Admin with Reader.
17. Do not couple the Reader directly with Amazon Polly.
18. Do not call Amazon Polly from the frontend.
19. Do not depend on connectivity for downloaded content.
20. Do not proceed to the next phase if critical deliverables are missing.

29. CODEX RESPONSE FORMAT IN EACH ITERATION

At the start of an iteration respond with:

CURRENT PROJECT STATE

Active phase:

Active task:

Task status:

Dependencies:

Files to inspect:

Files to modify:

Expected result:

Risks:

During implementation you must report:

IMPLEMENTATION PROGRESS

Completed:

In progress:

Blocked:

Problems found:

Decisions made:

At the end respond with:

SESSION SUMMARY

Phase:

Tasks completed:

Tasks still pending:

Files created:

Files modified:

Tests executed:

Test results:

Documentation updated:

Known issues:

Exact next task:

Recommended next command:

30. FIRST RUN

On the first run you must not immediately begin developing visual components or integrate Amazon Polly.

First you must:

1. analyze this prompt;
2. create the documentation structure;
3. create the phases;

4. create all tasks for Phase 0;
5. create the initial state;
6. create the decision log;
7. create the risk register;
8. create the initial requirements;
9. create the proposed architecture;
10. propose the implementation plan;
11. indicate any risk or inconsistency;
12. leave the first task ready to continue.

After that you must execute Phase 0 in an orderly manner.

Do not proceed to Phase 1 until Phase 0 meets all its acceptance criteria.

31. FINAL OBJECTIVE

FollowRead must become a professional portfolio project that demonstrates:

- product design;
- UX/UI;
- React;
- TypeScript;
- Python;
- FastAPI;
- PostgreSQL;
- Amazon Polly;
- Amazon S3;
- audio and text synchronization;
- mobile applications with Capacitor;
- offline mode;
- accessibility;
- testing;
- security;
- architecture;
- documentation;
- CI/CD;
- phased project management.

The code must be understandable for a student, maintainable by a junior developer, and sufficiently organized to be evaluated by a senior developer or a software architect.

Begin now with the prompt review, the creation of the tracking system, and the controlled execution of Phase 0.